VECTOR

MICROSAR Fr ERay

Technical Reference

Tricore

Version 1.42

Authors	Juergen Schaeffer, Mario Kunz, Sebastian Gärtner, Sebastian Schmar, Oliver Reineke, Roland Hocke, Matthias Müller
Status	Released

1 Document Information

1.1 History

Author	Date	Version	Remarks
Roland Hocke	2013-05-14	1.34	Creation and content copy from MSR3 document
	2013-06-26	1.35	Remove obsolete MTS
Roland Hocke	2014-01-14	1.36	Corrected Tricore Aurix Interrupt Config
Roland Hocke	2014-01-29	1.37	Buffer alignment hints
Matthias Müller	2015-02-20	1.38	Renaming of document name
Matthias Müller	2015-09-28	1.39	Description of Endinit and Protected Register Access
Matthias Müller	2015-11-27	1.40	ESCAN00080370 The usage of the Appl_TricoreAurixInit() function is not described in the TechRef SafeBSW limitations
Matthias Müller	2017-02-01	1.41	Added description of FEAT-2289: Dynamic configuration of PlatestTx (ApplFr_WriteCcReg)
Oliver Reineke	2018-04-16	1.42	Added description of FR_TRICORE_AURIX_IR_CONF

Table 1-1 History of the document

1.2 Reference Documents

No.	Title	Version
[1]	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	3.2.0
[2]	AUTOSAR_SWS_DiagnosticEventManager.pdf	4.2.0
[3]	TechnicalReference_Fr.pdf	1.0 or later
[4]	E-Ray_Errata_Sheet_20100215.pdf and later	REL20100215 and later
[5]	TechnicalReference_SchM.pdf	2.5 or later
[6]	TC1797 Target Specification	1.5
[7]	AURIX Safety Manual_rev1v4_2015-12-01.pdf	1.4

Table 1-2 Reference documents

1.3 Scope of the Document

This technical reference describes the specific use of the FlexRay ERay driver software. It supplements the general FlexRay driver technical reference [3].



Please note
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Document Information 2
1.1 History 2
1.2 Reference Documents 3
1.3 Scope of the Document 3

2 Component History 7

3 Introduction 8

4 Functional Description 9

5 Integration 10
5.1 Scope of Delivery 10
5.1.1 Static Files 10
5.1.2 Dynamic Files 10
5.2 Compiler Abstraction and Memory Mapping 10
5.3 Critical Sections 11
5.4 General Integration notes 12
5.4.1 Calculation of Timeout Loops 12
5.5 Integration notes for Infineon Tricore 12
5.5.1 ERay Base Address 12
5.5.2 FlexRay Memory Buffer number 13
5.5.3 FlexRay Memory Buffer size 13
5.5.4 EndInit Protection 13
5.5.5 Protected Register Access 13
5.5.6 Interrupt routines 13
5.5.7 Buffer alignment 14
5.5.8 Byte order 15
5.5.9 AURIX ECC trap 15

6 API Description 16
6.1 Type Definitions 16
6.2 Interrupt Service Routines provided by Fr ERay 16
6.3 Callout functions 18
6.3.1 Appl_UnlockEndinit 18
6.3.2 Appl_LockEndinit 18
6.3.3 ApplFr_WriteCcReg 19
6.4 Fr ERay specific function behavior 19

6.5 Services used by Fr ERay 19
6.6 Callback Functions 20
6.7 Configurable Interfaces 20
6.7.1 Notifications 20
6.7.2 Callout Functions 20

7 Configuration 21
7.1 Hardware Fifo 21
7.2 Configuration with DaVinci Configurator 5 21

8 AUTOSAR Standard Compliance 22
8.1 Deviations 22
8.1.1 Fr_GetSyncFrameList 22
8.1.2 Fr_GetPOCStatus 22
8.2 Additions/ Extensions 22
8.3 Limitations 22

9 Glossary and Abbreviations 23
9.1 Glossary 23
9.2 Abbreviations 23

10 Contact 24

Illustrations

Es konnten keine Einträge für ein Abbildungsverzeichnis gefunden werden.

Tables

Table 1-1 History of the document 2
Table 1-2 Reference documents 3
Table 2-1 Component history 7
Table 5-1 Static files 10
Table 5-2 Compiler abstraction and memory mapping 11
Table 6-1 Fr_IrqLine0/Fr_IrqLine1 16
Table 6-2 Fr_IrqTimer0 17
Table 6-3 Appl_UnlockEndInit 18
Table 6-4 Appl_LockEndInit 19
Table 6-5 Configuration dependent services used by Fr ERay 19
Table 9-1 Glossary 23
Table 9-2 Abbreviations 23

2 Component History

Component Version	New Features
4.00.00	Reimplementation of embedded code for SafeBSW (ASIL-D)
4.01.00	Support two separate FR controllers for AURIX

Table 2-1 Component history

3 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module Fr ERay that is specific for the communication controller E-Ray and the used micro controller. The basic functionality API architecture is already described within the document [3].

4 Functional Description

Please refer to [3].

© 2018 Vector Informatik GmbH

5 Integration

This chapter gives all necessary information for the integration of the MICROSAR Fr ERay into an application environment of an ECU. It describes ERay specific terms in addition to [3].

5.1 Scope of Delivery

The delivery of the Fr ERay contains the following additional files which are described in the chapters 5.1.1 and 5.1.2:

5.1.1 Static Files

File Name	Description
Fr_ERay.h	This file replaces the file Fr_<CC>.h listed in the document [3].

Table 5-1 Static files

5.1.2 Dynamic Files

No additional dynamic files. Please refer to [3].

5.2 Compiler Abstraction and Memory Mapping

The following table describes the memory section names and the compiler abstraction definitions of the Fr ERay and illustrates their assignment among each other.

Compiler Abstraction Definitions / Memory Mapping Sections	[Abstraction Name]	FR_CODE	FR_VAR_NOINIT	FR_VAR_NOINIT_FAST	FR_VAR_FRM	FR_CONST	FR_PBCFG	FR_APPL_CODE	FR_CODE_ISR
FR_START_SEC_CODE FR_STOP_SEC_CODE		■							
FR_START_SEC_VAR_NOINIT_UNSPECIFIED FR_STOP_SEC_VAR_NOINIT_UNSPECIFIED			■						
FR_START_SEC_VAR_FAST_NOINIT_UNSPECIFIED FR_STOP_SEC_VAR_FAST_NOINIT_UNSPECIFIED				■					

FR_START_SEC_CONST_UNSPECIFIED FR_STOP_SEC_CONST_UNSPECIFIED						■	■		
FR_START_SEC_PBCFG_ROOT FR_ STOP _SEC_PBCFG_ROOT							■		
FR_START_SEC_PBCFG FR_STOP_SEC_PBCFG							■		
FR_START_SEC_CONST_32BIT FR_STOP_SEC_CONST_32BIT						■	■		
FR_START_SEC_CONST_16BIT FR_STOP_SEC_CONST_16BIT						■	■		
FR_START_SEC_CONST_8BIT FR_STOP_SEC_CONST_8BIT						■	■		
FR_APPL_START_SEC_CODE FR_APPL_STOP_SEC_CODE								■	
FR_START_SEC_CODE_ISR FR_STOP_SEC_CODE_ISR									■

Table 5-2 Compiler abstraction and memory mapping

Caution
Please ensure that the define FR_PBCFG has the same value as FR_ CONST in configurations where “Pre-compile Configuration” or “Link-time Configuration” is selected.

5.3 Critical Sections

To ensure data consistency and proper functionality of the Fr ERay the exclusive area `FR_EXCLUSIVE_AREA_0` must be provided during the integration.

Considering the timing behavior of your system (e.g. depending on the CPU load of your system, priorities and interruptibility of interrupts and OS tasks and their jitter and delay times) the integrator must choose and use an exclusive area configuration which ensures that functions of the FR driver do not interrupt each other. You can find a set of rules below which describe whether an exclusive area is needed by the Fr ERay or not.

The `FR_EXCLUSIVE_AREA_0` should be used if at least one of the following rules hold true:

- It is possible that the function `Fr_TransmitTxLPdu` is interrupted itself. E.g. this could be the case, if the transmit request was triggered by the upper layer of the FrIf from task level (this could be the case if the attribute `FrIfImmediate` is set) and the execution is interrupted by the FrIf job list execution from interrupt level which also calls `Fr_TransmitTxLPdu` (for decoupled PDUs).
- If the feature “Reconfig LPdu Support” is enabled and it is possible that the functions `Fr_DisableLPdu` or `Fr_ReconfigLPdu` are interrupted by the function `Fr_TransmitTxLPdu` and vice versa.

The recommended implementation for the `FR_EXCLUSIVE_AREA_0` of the FR driver depends on execution context of the FrIf job list.

The `FR_EXCLUSIVE_AREA_0` shall disable/enable the FlexRay timer interrupt or call `SuspendAllInterrupts()` and `ResumeAllInterrupts()` in case the job list is executed from interrupt context.

The `FR_EXCLUSIVE_AREA_0` shall call `SuspendOSInterrupts()` and `ResumeOSInterrupts()` in case the FrIf job list is executed from task context. As an alternative, the task switch can be prevented by implementing an OS resource lock for the involved tasks.

5.4 General Integration notes

A workaround for errata #27 in [4] is implemented in module FrSm. If no FrSm is used, the application must implement a workaround.

5.4.1 Calculation of Timeout Loops

Please refer to [3] for a general description of this feature. This section describes the configuration of the worst-case execution time value for the ERay CC.

The worst-case execution time value depends on the configured “BCLK“ speed. The time value can be calculated as follows:

Worst-case execution time = 1/BCLK * 1329

For example:

Worst-case execution time = 1/64MHz * 1329 = 20,7 micro seconds

The maximum wait time can be configured with “Timeout Duration Factor” parameter. The recommended “Timeout Duration Factor” value in the example above is 380 in case the internal system clock of the micro controller is set to 128MHz.



Info

The timeout expiration is handled in a simple while loop. You can adapt the “Timeout Duration Factor” in case you use other BCLK or internal system clock values. The new value of the “Timeout Duration Factor” can be determined by measuring the execution time of the while loop.

```
while (timeoutCounter < "Timeout Duration Factor")
{
  timeoutCounter++;
}
```

5.5 Integration notes for Infineon Tricore

5.5.1 ERay Base Address

The base address of the ERay CC is set to the address that is mentioned in the hardware reference manual.

5.5.2 FlexRay Memory Buffer number

The ERay CC can address a maximum number of 128 message buffers.

5.5.3 FlexRay Memory Buffer size

The available FlexRay memory on ERay CC is 8kB of Message RAM.

5.5.4 EndInit Protection

The FR driver needs access to the EndInit protected registers, hence the application must provide the Appl_UnlockEndinit() and Appl_LockEndinit() functions. For implementation details, please refer to section Callout functions.

5.5.5 Protected Register Access

If the protected register access is configured by the OS, you need to create two areas that have the following two OS Identifier names:

FR_CLC_PROTECTED_AREA_ENDINIT

FR_IR_PROTECTED_AREA_ENDINIT

Those two areas must cover the address spaces of the CLC register and the interrupt registers of the IR.
Then writing those registers is done via OS APIs and not directly by the FR driver.

5.5.6 Interrupt routines

On the AUDO platform (TC17xx) the FR driver supports the configuration of the interrupt registers ERAY_INT0SRC and ERAY_TINT0SRC for eray_line0 and eray_timer0 interrupt routines. Refer to [6] for further details about the configuration of the TriCore interrupt priorities. The configuration of other interrupt specific service registers is not supported by the FR driver.

On the AURIX or AURIX+ platform (TC2xx and TC3xx) the FR driver doesn't handle the configuration of the Interrupt Router (IR). This is either done by MICROSAR OS Gen7 or must be ensured by the integrator for example by using the following glue code:



Example

```
#define  SRC_ERAYINT0        (*((volatile vuint32 *) ( 0xF0038BE0 )))
#define  SRC_ERAYTINT0       (*((volatile vuint32 *) ( 0xF0038BE8 )))
#define  HW_FRLINE0PRIO      7
#define  HW_FRTIME0PRIO      8

SRC_ERAYINT0  = 0x00000400 | HW_FRLINE0PRIO;
SRC_ERAYTINT0 = 0x00000400 | HW_FRTIME0PRIO;
```



Note
Due to backwards compatibility reasons between AUDO and AURIX the user can enable the Interrupt Router configuration (IR) via FR driver by using the following userconfig-file define on AURIX platform:

```
#define FR_TRICORE_AURIX_IR_CONF
```

Consider that this option is not allowed when using MICROSAR OS Gen7 and only the interrupt configuration of the ERAY0 is handled by the FR driver.

The interrupt functions can be called directly from OS or the BSP of the TriCore. The following the interrupt routines are provided in Fr_Irq.c by the FR driver:

- Fr_IrqLine0 or Fr_IrqLine1 can be selected by configuring the parameter “Line select for status interrupts”. By default Fr_IrqLine0 is used. This interrupt is only necessary if the ECUC Parameter FrCycleStartInterrupt is enabled. Otherwise it is not needed to enable this interrupt.
- Fr_IrqTimer0 calls the timer interrupt handler and the application callout function. The user can choose by configuration whether Fr_IrqTimer0 directly calls the application callout function or the FrIf job list execution.

5.5.7 Buffer alignment

The FlexRay buffers must be aligned to 16 bit addresses. This is important for all FlexRay frames with PDUs where `FrIfImmediate` is enabled. If `FrIfImmediate` is disabled, the FrIf automatically aligns the buffers to 32 bit.

If it is not possible to align the message buffers of frames with immediate PDUs, the FlexRay driver can align unaligned message itself. In this case the FlexRay driver needs more runtime for the message buffer access because it needs to check every messages alignment. This option can be enabled with a userconfig-file define:



Example

```
#define FR_NONALIGNED_SRCMEMORY
```



Caution
The `FR_NONALIGNED_SRCMEMORY` is not allowed in the SafeBSW use case.

5.5.8 Byte order

The Fr ERay module is compiled and tested with Little Endian Byte format.

5.5.9 AURIX ECC trap

Due to the erratum Flexray_TC.H002 (alias names are Flexray_TC.H001 and Flexray_AI.H010) the ECC error notification must be cleared before calling Fr_Init when ECC RAM check is enabled. If this function is not implemented correctly, an ECC trap may occur. An example implementation is provided by Infineon.



```
Void Appl_TricoreAurixInit();
```

6 API Description

6.1 Type Definitions

The Fr ERay does not define specific type definitions.

6.2 Interrupt Service Routines provided by Fr ERay

For general description please refer to [3] and the respective chapters under “Functional Description”.

Prototype	
`FUNC(void, FR_CODE) Fr_IrqLine0/Fr_IrqLine1 ()`	
Parameter	

Return code	

Functional Description	
Executes the status interrupts of ERay. By default cycle start interrupt are used by Fr ERay to synchronize Application with callback function `ApplFr_ISR_CycleStart`.	
Particularities and Limitations	
> Particularities, limitations, post-conditions, pre-conditions > An interrupt splitting is used. Fr ERay is initialized and interrupts are enabled. > Configuration attribute “Cycle Start Interrupt” or ECUC Parameter FrCycleStartInterrupt is enabled.	

Table 6-1 Fr_IrqLine0/Fr_IrqLine1

Prototype	
`FUNC(void, FR_CODE) Fr_IrqTimer0 ()`	
Parameter	

Return code	

Functional Description
Executes the Timer0 interrupt of ERay. By default this interrupt is used by Fr ERay. It also disables the interrupt.
Particularities and Limitations
> Particularities, limitations, post-conditions, pre-conditions > A separate interrupt for Timer0 is called by hardware and interrupt splitting is used. Fr ERay is initialized and interrupts are enabled.

Table 6-2 Fr_IrqTimer0

6.3 Callout functions

Within Fr_Init() and Fr_ControllerInit() the FlexRay driver configures the ERay clock register (ERAY_CLC) and depending on the platform the interrupts of the Interrupt Router (IR). The user must guarantee full write access to these registers, which are protected by the watchdog-control-register. The user must provide the following functions to disable (`Appl_UnlockEndinit`) and re-enable (`Appl_LockEndinit`) the write protection by accessing the watchdog register. Please refer to Application Note *AN-ISC-8-1172_TriCore_ENDINIT_protection_handling.pdf* for further details.

6.3.1 Appl_UnlockEndinit

Prototype	
`FUNC(void, FR_APPL_CODE) Appl_UnlockEndinit(void)`	
Parameter	

Return code	

Functional Description	
Called to unlock protected registers in ERay cell.	
Particularities and Limitations	
> This function can be left empty if the Tricore is in Supervisor mode and can access the ERAY_CLC even if it is locked.	

Table 6-3 Appl_UnlockEndInit

6.3.2 Appl_LockEndinit

Prototype	
`FUNC(void, FR_APPL_CODE) Appl_LockEndinit(void)`	
Parameter	

Return code	

Functional Description	
Called to lock protected registers in ERay cell again.	
Particularities and Limitations	
> This function can be left empty if the Tricore is in Supervisor mode and can access the ERAY_CLC even if it is locked.	

Table 6-4 Appl_LockEndInit

6.3.3 ApplFr_WriteCcReg

Prototype	
`FUNC(void, FR_APPL_CODE) ApplFr_WriteCcReg (uint8 Fr_CtrlIdx)`	
Parameter	
Fr_CtrlIdx	Fr_CtrlIdx is a zero-based index identifies the CC controller for which the API call has to be applied.
Return code	

Functional Description	
This function is called in Fr_ControllerInit after completion of the initial static configuration. It can be used to modify a CC register, e.g. the pLatestTx setting of the FlexRay controller.	
Particularities and Limitations	
> The changes made by this function are not checked by the CC readback support (FrIfReadCCConfigApi) > This function is only called if preprocessor switch FR_OVERWRITE_REGISTER is STD_ON. This switch can only be enabled via user config file.	

6.4 Fr ERay specific function behavior

The FR driver for ERay CCs provides all the general services which are described in [3] but the following APIs differ from standard behavior:

> Fr_EnableAbsoluteTimerIRQ & Fr_DisableAbsoluteTimerIRQ:

  Fr ERay driver does not allow to enable or disable the timer interrupts. The application has to enable or disable the ERay timer interrupt in the platform specific interrupt controller.

6.5 Services used by Fr ERay

General services are described in [3].

Depending on configuration services used by Fr ERay are listed in next table. For details about prototype and functionality refer to the documentation of the providing component.

Configuration/Precondition	Component	API
Application callback at timer 0 interrupt and BSM FrIf is used	FrIf	FrIf_JobListExec_0

Table 6-5 Configuration dependent services used by Fr ERay

6.6 Callback Functions

There is no Fr ERay specific callback function available.

6.7 Configurable Interfaces

6.7.1 Notifications

There is no Fr ERay specific notification function available.

6.7.2 Callout Functions

There is no Fr ERay specific callout function available.

7 Configuration

In the Fr ERay the parameters can be configured with the following methods:

> Configuration in DaVinci Configurator 5; for a detailed description see 7.2.

7.1 Hardware Fifo

This platform does not support Hardware FIFO support.

7.2 Configuration with DaVinci Configurator 5

The Fr ERay is configured with the help of the configuration tool DaVinci Configurator 5. For general configuration information please refer to [3].

8 AUTOSAR Standard Compliance

8.1 Deviations

For general deviations please refer to [3] and [4].

8.1.1 Fr_GetSyncFrameList

ERay CC always receives a transmitted SYNC frame on both channels, even if a channel is not used. The transmission of a SYNC frame depends on configuration.

8.1.2 Fr_GetPOCStatus

The value for Fr_POCStatusPtr->SlotMode is derived out of the ERay bit CCSV.SLT.

On ERay versions higher than 1.0.1 the ERay Bit CCSV.SLT (single slot mode) cannot be read in Poc states: “Config, Default Config, Halt” and always return 0. In all other POC states the Value is according configuration. For a detailed description please refer to [4].

8.2 Additions/ Extensions

Please refer to [3].

8.3 Limitations

Please refer to [3].

> The user must configure the output driver strength and the slew rate for the TriCore Pads. This is normally done by configuring the Pad Driver Mode Registers (see Tricore target specification for more info).

For the hardware limitations described in [4] configuration warnings with solving suggestions are displayed in the config tool.

9 Glossary and Abbreviations

9.1 Glossary

Term	Description
EAD	Embedded Architecture Designer; generation tool for MICROSAR components
CFG5	DaVinci Configurator 5. Configuration tool for MICROSAR 4

Table 9-1 Glossary

9.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
CC	FlexRay Communication Controller
E-Ray	Specific implementation of a FlexRay Communication Controller
DBA	Direct Buffer Access
DEM	Diagnostic Event Manager
DET	Development Error Tracer
EAD	Embedded Architecture Designer
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
PPort	Provide Port
RPort	Require Port
RTE	Runtime Environment
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 9-2 Abbreviations

10 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector-informatik.com